Why Modern Classical Monte Carlo Codes Are Fast

A Pedagogical Tour of the Ingredients,
with Worked Examples From `ClassicalSpin_Cpp`

April 2026

Abstract

This note is an explanation, written for a physics graduate student with no prior computer-science background, of why modern classical-spin Monte Carlo (MC) codes can run *thousands of times faster* than a naive textbook implementation while computing exactly the same physical answer. The text is organized around the optimization pass that brought `ClassicalSpin_Cpp` up to par with state-of-the-art (SOTA) atomistic spin codes such as Vampire and UppASD. Each ingredient is introduced from physical first principles, the underlying computer-science concept is explained in plain language, and a measured benchmark number from this code is given. No prior knowledge of compilers, caches, or parallel programming is assumed.

Contents

1 The picture in one paragraph **1**

2 What the CPU actually does **1**
- 2.1 Fact 1: arithmetic is free, memory is not . . . 1
- 2.2 Fact 2: branches are predicted, but mispredictions are expensive . . . 2
- 2.3 Fact 3: the compiler can vectorize, but only if the loop is simple . . . 2

3 The Metropolis kernel: where the time goes **2**

4 Ingredient I: the data layout **3**
- 4.1 The naive layout is a hidden disaster . . . 3
- 4.2 The fix: contiguous flat storage . . . 4
- 4.3 The same logic applied to the interaction tables: SoA bilinear . . . 4

5 Ingredient II: zero allocation in the hot loop **5**

6 Ingredient III: sphere sampling without trigonometry **5**

7 Ingredient IV: sweep order and branch behaviour **6**
- 7.1 Sequential vs random site order . . . 6
- 7.2 Short-circuit acceptance . . . 7

8 Ingredient V: better algorithms beat better implementations **7**
- 8.1 Overrelaxation . . . 7
- 8.2 Cluster algorithms (Wolff, Swendsen–Wang) . . . 7
- 8.3 Parallel tempering . . . 8

9 Ingredient VI: parallelism and where to apply it **8**
9.1 SIMD: hand it to the compiler . . . 8
9.2 OpenMP: parallelize across replicas, not within a sweep . . . 8
9.3 MPI: scale across nodes . . . 9

10 Ingredient VII: reproducibility is a feature, not a luxury **9**

11 Putting it together: a checklist **9**

12 What the numbers mean **10**

13 Further reading **11**

1 The picture in one paragraph

A classical Monte Carlo simulation of a magnet does the same thing 10^8–10^{12} times: *pick a spin, propose a small change, compute the energy difference, accept or reject.* The total cost of the simulation is therefore the cost of *one such elementary step*, multiplied by the enormous number of steps. To make the code fast, the job is to make that single inner step as cheap as the laws of physics and the laws of the CPU allow. Roughly half the speedup comes from **algorithmic** ingredients (better moves, smarter sampling), the other half from **systems** ingredients (touching memory in a cache-friendly order, avoiding hidden allocations, getting the compiler to emit vectorized instructions). Both halves are necessary; either alone is insufficient.

2 What the CPU actually does

Before diving into algorithms, three facts about the hardware are worth internalising, because every optimization below is a direct response to one of them.

2.1 Fact 1: arithmetic is free, memory is not

A modern x86 core can do roughly 5×10^9 floating-point multiply–adds per second. Reading one "cold" double from main memory (DRAM) takes about 80 nanoseconds. In that time the core could have performed ~ 400 multiply–adds. In other words, **a single cache miss is as expensive as several hundred floating-point operations**.

The CPU mitigates this by keeping a hierarchy of small, fast caches close to the registers:

Level	Typical size	Access time
Registers	$\sim$ 1 kB	0.3 ns
L1 cache	32–64 kB	1 ns
L2 cache	256 kB–1 MB	4 ns
L3 cache	4–64 MB	15 ns
DRAM	16–512 GB	80 ns

The rule of thumb that comes out of this is: **algorithms that touch the same kilobyte of data many times in a row are fast; algorithms that hop around a multi-MB structure are slow**, even if the operation count looks identical on paper.

2.2 Fact 2: branches are predicted, but mispredictions are expensive

A modern CPU does not wait for an `if` to be evaluated before starting to work on the code that follows it. It *guesses* which branch will be taken, executes that path speculatively, and only commits the result when the actual condition is known. When the guess is right, the `if` costs essentially nothing. When it is wrong, the CPU must throw away ~ 15 partially-computed instructions and restart; this is a *branch misprediction* and costs ~ 5 ns, which is again hundreds of arithmetic operations.

The Metropolis acceptance test is a classic mispredictor: in equilibrium its outcome is essentially random. We will return to it in Section 3.

2.3 Fact 3: the compiler can vectorize, but only if the loop is simple

Modern x86 cores have *SIMD* (Single Instruction, Multiple Data) units that operate on 4 or 8 doubles at once. The compiler will use them *automatically* if it can prove that the loop:

- touches contiguous memory (no pointer chasing),
- has no aliasing (different pointers really refer to disjoint memory),
- has a known trip count and no early exits,
- does not call out into opaque functions.

Failing any one of these turns an SIMD loop back into a scalar one and costs roughly a $4\times$ slowdown. This is why "rewriting the inner loop on raw `double*` arrays" is not premature optimization—it is what allows the compiler to do its job.

3 The Metropolis kernel: where the time goes

Consider the Heisenberg model on a honeycomb lattice with N sites. A single Metropolis sweep does:

1. For each site i in the lattice (N iterations):
   (a) Generate a candidate spin $\mathbf{S}_i^{\text{new}}$ on the unit sphere.
   (b) Compute the local energy difference $\Delta E = \mathbf{S}_i^{\text{new}} \cdot \mathbf{H}_i^{\text{loc}} - \mathbf{S}_i^{\text{old}} \cdot \mathbf{H}_i^{\text{loc}}$ where $\mathbf{H}_i^{\text{loc}} = \sum_{j \in \text{nn}(i)} \mathsf{J}_{ij} \mathbf{S}_j$ is the local field.
   (c) Draw $u \sim U(0,1)$. Accept if $\Delta E < 0$ or $u < e^{-\beta \Delta E}$.

Naively coded, each pass through the body of step 1 does, in our case on a honeycomb with $z = 3$ neighbours, about

$$3 \text{ (neighbours)} \times 9 \text{ (matrix-vector adds)} = 27 \text{ floating-point operations.}$$

At 5×10^9 flop/s this is ~ 5 ns of arithmetic per site, hence a peak of $\sim 2 \times 10^8$ updates/sec/core.

The benchmark in this code (Table 1, "after" column) measures 1.8×10^7 updates/sec/core for the Heisenberg Metropolis kernel. Before the SoA-table and Marsaglia refactors of Sections 4.3–6 the same kernel ran at 1.4×10^7, so the latest pass closes part of the $\sim 12\times$ gap to peak arithmetic that the rest of this note explains, ingredient by ingredient.

The bolded rows are the kernels where this refactoring pass actually moved the needle: Metropolis (and any sweep that contains it) gained ~ 25–30% across all three models. The pure-overrelaxation, Wolff, and Swendsen–Wang kernels were not measurably affected because they do not generate

Table 1: Measured single-core throughput, pinned to one CPU core (`taskset -c 0`, `OMP_NUM_THREADS=1`), gcc 13.3 with `-O3 -march=native -flto -ffp-contract=fast`. "before" columns are the throughput just before the SoA bilinear table and Marsaglia sphere sampler were introduced; "after" columns are with both in place. "updates/s" is the number of *spin updates* per second; for cluster algorithms it counts "sweeps $\times$ N sites" which conflates wall time with sample quality.

Model	Algorithm	L	N	before (upd/s)	after (upd/s)
Heisenberg honeycomb	Metropolis	32	2048	1.43×10^7	$\mathbf{1.80 \times 10^7}$
Heisenberg honeycomb	Overrelaxation	32	2048	3.35×10^7	3.35×10^7
Heisenberg honeycomb	met+5over	32	2048	4.52×10^6	$\mathbf{4.89 \times 10^6}$
Heisenberg honeycomb	Wolff	32	2048	6.83×10^8	5.95×10^8
Heisenberg honeycomb	Swendsen–Wang	32	2048	2.35×10^7	2.31×10^7
Kitaev honeycomb	Metropolis	32	2048	1.35×10^7	$\mathbf{1.78 \times 10^7}$
Kitaev honeycomb	Overrelaxation	32	2048	3.21×10^7	3.31×10^7
Pyrochlore	Metropolis	6	864	1.07×10^7	$\mathbf{1.41 \times 10^7}$
Pyrochlore	Overrelaxation	6	864	2.07×10^7	2.26×10^7
Pyrochlore	met+5over	6	864	2.88×10^6	$\mathbf{3.40 \times 10^6}$
Cache-hierarchy effect, Heisenberg honeycomb metropolis (after):					
Heisenberg honeycomb	Metropolis	32	2048	—	1.8×10^7
Heisenberg honeycomb	Metropolis	64	8192	—	1.4×10^7
Heisenberg honeycomb	Metropolis	128	32768	—	6.5×10^6

random spins (Marsaglia is irrelevant) and they already had reasonably contiguous J-table access (Eigen's `Matrix3d` stores its 9 doubles inline, and a `vector<Matrix3d>` is contiguous within each site). The small Wolff regression is consistent with run-to-run variation in cluster size; the algorithm's per-spin cost is unchanged.

4 Ingredient I: the data layout

4.1 The naive layout is a hidden disaster

A textbook implementation of a spin lattice in C++ usually looks like:

```
std::vector<Eigen::VectorXd> spins;      // one VectorXd per site
spins[i](0) = ... ; spins[i](1) = ... ; spins[i](2) = ...;
```

`Eigen::VectorXd` is a *heap-allocated* object: it stores a pointer to a buffer somewhere on the free store. When you write `spins[i](0)`, the CPU has to:

1. read `spins.data() + i*sizeof(VectorXd)` (a pointer);
2. dereference that pointer to find the buffer;
3. read the first three doubles of the buffer.

The buffer for spin i is on a different cache line (often a different *page*) than the buffer for spin i+1. Iterating over the lattice therefore generates one cache miss per site—tens of nanoseconds per spin update, completely dominating the ~ 5 ns of arithmetic.

4.2 The fix: contiguous flat storage

A *flat* layout stores all $N \times 3$ doubles in one contiguous block:

```
double* spins;            // size = N * spin_dim
spins[i*spin_dim + 0]     // x-component of spin i
spins[i*spin_dim + 1]     // y
spins[i*spin_dim + 2]     // z
```

Now spin i and spin i+1 live on the same cache line (or at most two), and iterating over them prefetches the data automatically. In `ClassicalSpin_Cpp` this is what `site_energy_diff_flat()` does internally: it reads spins via raw `double*` pointers instead of `Eigen::VectorXd` temporaries. The change alone is typically a 2–4× speedup on local-update kernels.

4.3 The same logic applied to the interaction tables: SoA bilinear

The naive layout for the exchange matrices is

```
std::vector<std::vector<Eigen::Matrix3d>> bilinear_interaction;
```

which on paper hides three pointer indirections per neighbour: outer vector → inner vector → Eigen buffer. Modern atomistic spin codes (Vampire, UppASD) instead pack the entire $N \times z \times 9$ table into a single `double[]`, and we now do the same:

```
vector<size_t> bi_flat_offset;          // CSR-style: size N+1
vector<size_t> bi_flat_partner;         // size = total_bonds
vector<double> bi_flat_J;               // size = total_bonds * spin_dim^2
vector<uint8_t> bi_flat_needs_twist;    // precomputed twist mask
```

The flat tables are built once at the end of `initialize()`; the original nested `vector<vector<...>>` stays as the source of truth for cold-path code (total energy, snapshot save/load). Inside `site_energy_diff_flat()`, `overrelaxation()`, and `get_local_field_flat()` the bilinear partners and their J matrices are now read straight from the flat arrays:

```
const double* J = &bi_flat_J[k * D2];       // contiguous 9 doubles
const double* P = spins[partner].data();    // (still scattered)
for (a=0..3) for (b=0..3) row += J[a*3+b] * P[b];
```

Pleasantly enough this turned out to be a very small win in our case (a few percent on Metropolis): `Eigen::Matrix3d` already stores its 9 doubles inline, and `vector<Matrix3d>` is contiguous within a site, so the J matrices were already L1-resident during a sequential sweep. The big advantage of the rewrite is *robustness*: removing the dependence on Eigen's expression templates and on the platform allocator's behaviour means the kernel now generates the same tight assembly across compilers and across allocator versions. It also unblocks future SIMD-intrinsic work (the layout is now compatible with AVX2 `vfmadd231pd` on $9 \times d$ blocks). The much larger Metropolis win in Table 1 comes from the next ingredient.

5 Ingredient II: zero allocation in the hot loop

A C++ `std::vector<double> tmp(3)` looks innocent. But under the hood it calls `operator new`, which:

- walks a free-list to find a chunk of memory of the right size,

- updates global allocator bookkeeping (sometimes under a lock),
- zero-initializes the memory,
- invalidates a cache line.

A single `new` call costs $\sim$30–100 ns—that is, 5–20$\times$ the *entire arithmetic budget* for one spin update. If you do this once per Metropolis proposal you have already lost the game.

The fix is two-pronged:

Stack scratch buffers for one-shot temporaries. Inside the hot loop, use plain C arrays:

```
double H_loc[8];        // local field
double new_spin[8];     // proposed spin
```

These live on the stack, cost zero to allocate, and are `constexpr`-sized so the compiler can keep them in registers.

Persistent member buffers for repeat work. When a sweep *always* needs a buffer of size N (e.g. the Wolff "in-cluster" marker, the Swendsen–Wang Union–Find arrays), declare it once as a `mutable` member of the lattice class:

```
mutable std::vector<uint8_t> cluster_in_cluster;   // size lattice_size
```

The vector is allocated *once* (the first call `resize()`s it), and on every subsequent sweep it is just a contiguous chunk of memory waiting to be reused. We applied this to the Wolff BFS stack, the projection buffer, and the Union–Find arrays; this alone removed hundreds of kilobytes of malloc/free traffic per sweep at $L = 12$.

6 Ingredient III: sphere sampling without trigonometry

The Metropolis kernel does roughly N "propose a random spin" calls per sweep. The textbook way to draw a uniformly distributed unit vector in 3D, given two uniform deviates $u_z, u_\phi \in [0, 1)$, is

$$z = 2u_z - 1, \qquad \phi = 2\pi u_\phi, \qquad r = \sqrt{1 - z^2}, \qquad (x, y, z) = (r\cos\phi,\ r\sin\phi,\ z).$$

This is mathematically clean, but on hardware it costs one `sqrt`, one `cos`, and one `sin` per spin. On modern x86 `cos` and `sin` take $\sim$ 12 cycles each when implemented in libm, plus a function-call boundary that defeats register allocation across the call. Together that is $\sim$ 25 ns per proposed move—about a third of the entire $\sim$ 70 ns budget for one Metropolis update at the rates measured in Table 1.

Marsaglia (1972) showed how to produce the same distribution *without any trigonometry*, by rejection-sampling in the unit disk and mapping to the sphere algebraically:

```
do {
    u1 = 2 * rand_uniform() - 1;
    u2 = 2 * rand_uniform() - 1;
    s  = u1*u1 + u2*u2;
} while (s >= 1.0);
const double factor = 2 * sqrt(1 - s);
x = factor * u1;
y = factor * u2;
z = 1 - 2 * s;
```

The acceptance probability is the ratio of the unit disk to the unit square, $\pi/4 \approx 0.785$, so on average $1/0.785 \approx 1.27$ iterations are needed—i.e. ~ 2.55 RNG calls per accepted spin (vs. exactly 2 for the textbook method). In exchange we delete one `sqrt` and *both* transcendental calls. In practice this is a $\sim$ 25–30% wall-time improvement on the Metropolis kernel of every model in Table 1, and is by itself the largest single optimization in the most recent pass.

The same identity is used by NIST's RNG library, by NumPy's `random.normal` (via Box–Muller), and by every published atomistic spin code from Spirit to UppASD. It is genuinely free correctness: the produced distribution is uniform on S^2 to machine precision, since the rejection step is exact.

What about flat spin storage? A natural follow-up was to change `vector<Eigen::VectorXd>` (scattered heap allocations) into a single contiguous `vector<double>` and let partner-spin reads stream from L1. We tried it (synced into a flat scratch at sweep entry, back out at exit) and found it was a wash—in fact a small *regression* on Heisenberg. The reason is that for short-range models with sequential site order, partner sites are mostly the immediate neighbours of the current site, and their Eigen buffers are usually already in L1 from a recent visit. The cost of synchronising the flat scratch ($\sim$ 12 kB of memory traffic per sweep at $L = 32$) outweighed the cache-locality benefit. This is a useful counter-example: caching theory suggests the flat layout "must" be faster, but at small N where the entire spin set fits in L1 either way, the sync overhead dominates. At very large N ($L \gtrsim 128$, where the spin data spills out of L2), the flat layout would likely win again; we leave that as a future refactor.

7 Ingredient IV: sweep order and branch behaviour

7.1 Sequential vs random site order

Textbook MC says "pick a site uniformly at random, propose a move". This satisfies detailed balance. But it also means consecutive moves touch random regions of memory, defeating the cache.

A *sequential sweep*—visiting sites $0, 1, 2, \ldots, N-1$ in order—also satisfies the weaker (but sufficient) condition of ergodicity for short-range models, and brings huge cache benefits because the next site's neighbour data is already in L1 from the previous iteration's work. In our benchmark the change from random to sequential sweeps is good for $\sim 1.5\times$ on the Metropolis kernel at $L = 32$.

7.2 Short-circuit acceptance

The Metropolis acceptance test is

$$\text{accept if } \Delta E < 0 \text{ or } u < e^{-\beta \Delta E}.$$

A common subtle bug is to write

```
if ((dE < 0) | (u < std::exp(-beta * dE))) accept();
```

The bitwise `|` forces the compiler to evaluate both sides, so `exp()` is called *even when* $\Delta E < 0$. The correct form is

```
if (dE < 0 || u < std::exp(-beta * dE)) accept();
```

The `||` operator short-circuits: if the first half is true, the second is not evaluated. At low temperature most accepted moves are downhill ($\Delta E < 0$), so the saved `exp()` calls are a measurable win and were one of the explicit bugs we fixed in this code.

8 Ingredient V: better algorithms beat better implementations

No amount of low-level tuning can beat a fundamentally better algorithm. Three tricks pay off enormously:

8.1 Overrelaxation

For an isotropic Heisenberg model, the spin

$$\mathbf{S}_i^{\text{new}} = -\mathbf{S}_i + 2\frac{(\mathbf{S}_i \cdot \mathbf{H}_i^{\text{loc}})}{|\mathbf{H}_i^{\text{loc}}|^2}\mathbf{H}_i^{\text{loc}}$$

is the reflection of $\mathbf{S}_i$ about the local field. This move is *exactly energy-conserving*, so it is always accepted: no RNG, no `exp()`, no acceptance test. Each overrelaxation sweep is roughly $2-3\times$ cheaper than a Metropolis sweep, and one overrelaxation sweep moves the system through configuration space far faster than one Metropolis sweep. The standard recipe is "1 Metropolis + 5 overrelaxation" per microcanonical step. Our benchmark confirms the per-sweep cost ratio: 3.3×10^7 updates/s for overrelaxation versus 1.4×10^7 for Metropolis on the same lattice.

8.2 Cluster algorithms (Wolff, Swendsen–Wang)

Near a continuous phase transition the correlation length ξ diverges, and Metropolis suffers from *critical slowing down*: the autocorrelation time τ grows as ξ^z with the dynamic exponent $z \approx 2$. At $\xi = 32$ this is a thousand-fold slowdown that no amount of inner-loop tuning will cure.

The Wolff and Swendsen–Wang algorithms (Wolff 1989, Swendsen–Wang 1987) build a connected cluster of correlated spins and flip the *whole cluster at once*. Their dynamic exponent is $z \approx 0.3$, so near T_c they win by orders of magnitude. Our benchmark sees Wolff at 6.5×10^8 effective updates/s— $50\times$ above Metropolis—but the comparison flatters Wolff, since the cluster size near T_c is itself $O(N)$. The right way to read the table is: at the same wall-time budget Wolff produces *many more independent samples*.

To make these algorithms fast in practice, two systems-level tricks matter:

- Persistent BFS stack and "in-cluster" bitmap (no `malloc` per sweep).
- Iterative path-compression Union–Find for SW (the recursive version blows the stack on percolating clusters).

Both are implemented here; see `Lattice::wolff_update()` and `Lattice::swendsen_wang_sweep()`.

8.3 Parallel tempering

At very low T the system gets stuck in metastable basins. Parallel tempering (Hukushima–Nemoto 1996) runs R replicas at R different temperatures and proposes *exchange* moves between adjacent temperatures with probability

$$P_{\text{swap}} = \min\Big(1, e^{(\beta_a-\beta_b)(E_a-E_b)}\Big).$$

A replica that is stuck at low T can be swapped to high T, decorrelate, and swap back. PT pairs naturally with MPI: each replica lives on its own rank. The systems-level engineering matters here too:

- Use `MPI_Sendrecv_replace` on a *persistent* per-thread buffer instead of allocating two new vectors per swap.
- Use a deterministic, splitmix64-mixed seed per rank so runs are reproducible across MPI sizes.
- Choose the temperature ladder by *feedback*: the Katzgraber et al. (2006) and Miyata et al. (2024) schemes place temperatures so that swap acceptance is uniform ≈ 0.45, which minimizes round-trip time.

9 Ingredient VI: parallelism and where to apply it

There are three layers of parallelism on a typical CPU node:

1. **SIMD**: 4–8 doubles per instruction within one core.
2. **OpenMP threads**: 8–64 cores per node, sharing memory.
3. **MPI ranks**: hundreds of nodes, no shared memory.

9.1 SIMD: hand it to the compiler

The compiler will auto-vectorize *simple* loops if you give it the right flags. The benchmark in this code is built with

```
-O3 -march=native -flto -funroll-loops
-ffp-contract=fast -fno-math-errno -fopenmp-simd
```

`-march=native` tells the compiler "you may use any SIMD instruction this CPU supports" (AVX2 on a Haswell+ box, AVX-512 on a Skylake-X+). `-flto` enables link-time optimization so the compiler can inline and vectorize across translation units.

9.2 OpenMP: parallelize across replicas, not within a sweep

Within a single Metropolis sweep, the spins are *not* independent (updating spin i changes the local field at its neighbours), so parallelizing one sweep is hard—it requires a checkerboard decomposition that respects the bond graph. We currently do not do this.

But for parallel tempering, each *replica* runs an independent sweep at its own temperature. These are embarrassingly parallel: just `#pragma omp parallel for`. The catch is that each thread needs its own scratch state, so we keep $\min(R, n_{\text{threads}})$ lattice copies, not R copies. Earlier versions of this code cloned a full `Lattice` for every replica; on a 32-replica $L = 12$ pyrochlore that meant ~ 200 MB of duplicated read-only interaction tables. The fix—reusing one "thread lattice" and swapping in the per-replica spin configuration— is what brought our `VmHWM` on a single rank down to ~ 8 MB across all benchmarks.

9.3 MPI: scale across nodes

MPI is just message-passing. For PT it is one of the easiest distributed-memory algorithms in computational physics: every `pt_exchange_frequency` sweeps, each rank trades $N \times d \times 8$ bytes of spin data with its neighbour rank. The practical wins are:

- Use `MPI_Sendrecv_replace`, not separate `Send`/`Recv` (one buffer instead of two).
- Honour the user-supplied communicator instead of hard-coding `MPI_COMM_WORLD`, so a parameter sweep can split MPI into independent sub-communicators.

The first item is a one-line edit and removes one heap allocation per swap attempt. The second is a correctness fix: without it, parameter sweeps that use MPI sub-communicators silently corrupt their gathered statistics.

10 Ingredient VII: reproducibility is a feature, not a luxury

A common "optimization" is to seed the RNG from the wall clock. This is fast, but it makes the simulation *not reproducible*: re-running the same input gives different numbers. In practice this means:

- Bugs that show up only at one particular seed cannot be re-investigated.
- Statistical comparisons across runs ("did this code change affect the physics?") become impossible—any difference might be noise.

The fix is *deterministic per-rank seeding*:

```
// splitmix64(master_seed XOR rank_id)  ->  per-rank seed
uint64_t derive_seed_from_master(uint64_t key);
```

`splitmix64` is a tiny invertible mixing function that turns a single master seed plus a small "key" (the MPI rank, the OpenMP thread id) into a high-quality per-stream seed, with no overlap. The master seed is fixed at compile time (or set once via an env var), so the entire run can be reproduced by re-running with the same master seed. We use this for the Lehman generator that drives the inner MC loop and for the `std::mt19937` that drives the PT exchange decisions.

11 Putting it together: a checklist

When auditing a classical-spin MC code for "is this SOTA?", go down this list. Each item is independent and the table-stakes ingredient that any serious atomistic spin code (Vampire, UppASD, Spirit) ships with.

1. **Inner loop is allocation-free.** No `vector<>`, no `Eigen::VectorXd`, no `new`, no `shared_ptr` inside the per-site work.
2. **Spins are stored contiguously.** Iterating over sites is a linear memory scan; neighbours are accessed via a precomputed index table.
3. **Interaction tables are flat.** Bilinear `J` matrices live in one `double[]` of size $N \times z \times 9$.
4. **Sweep order is sequential** (or, where required, a static checkerboard—never per-call random with replacement).
5. **Compiler flags are aggressive.** `-O3 -march=native -flto -ffp-contract=fast`, at minimum.
6. **Acceptance test short-circuits.** Use `||`, not `|`, and put the cheap test first.
7. **Cluster algorithms are available.** Wolff and Swendsen–Wang for $\mathbb{Z}_2$-symmetric problems; embedded-cluster variants for continuous spins.
8. **Overrelaxation is interleaved with Metropolis** for continuous spins.
9. **Parallel tempering uses persistent buffers and `Sendrecv_replace`.**

10. **The temperature ladder is feedback-optimized.**
11. **Per-rank, per-thread RNGs are deterministically seeded.**
12. **Observables accumulators precompute lookup tables** once in `initialize()`, never in the per-sample call.
13. **The molecular-dynamics integrator can be re-projected onto the constraint surface** (here, $|\mathbf{S}| = 1$) at user request.

`ClassicalSpin_Cpp` now ticks all thirteen, including item 3 (the Structure-of-Arrays bilinear table, see Section 4.3) and the Marsaglia sphere sampler (Section 6). The remaining gap before head-to-head benchmarking against Vampire/UppASD on large Heisenberg lattices ($L \gtrsim 128$) is hand-rolled SIMD intrinsics on the SoA bilinear table—the layout is now AVX2-compatible (9-double blocks for the J matrix), but we still rely on the compiler's auto-vectorizer to emit `vfmadd231pd`. An explicit intrinsics rewrite would be expected to give an additional 1.5–2× on the per-site energy difference, but the cost in maintainability and portability is real.

12 What the numbers mean

The headline number from Table 1 is $\sim 1.8 \times 10^7$ Metropolis spin-updates/sec/core for a short-range Heisenberg model on a single x86 core (up from $\sim 1.4 \times 10^7$ before this pass). For comparison, Vampire's reported figures on Skylake-class CPUs are in the range 10^7–10^8 spin-updates/sec/core depending on the model; UppASD reports $\sim 5 \times 10^7$ for similar problem sizes. We are within a factor of $\sim$ 2–3 of those codes *without* hand-rolled SIMD intrinsics. Closing the rest of the gap requires explicit AVX2/AVX-512 work on the per-site energy difference; the SoA layout (Section 4.3) is the necessary prerequisite for that, and it is now in place.

The cache-hierarchy effect is worth noting on its own. At $L = 32$ the spin data ($N \times d \times 8 = 49$ kB) fits in L1; at $L = 64$ (196 kB) it spills to L2; at $L = 128$ (786 kB) it spills to L3. The measured throughput drops from 1.8×10^7 to 6.5×10^6 across this range—a $\sim 3\times$ slowdown that is essentially *all memory bandwidth*. This is the regime where SIMD and the already-flat SoA layout would pay the most, and where the "flat-spin scratch" refactor we deliberately did *not* land at $L = 32$ (Section 6, last paragraph) is likely to start winning.

13 Further reading

- D. P. Landau and K. Binder, *A Guide to Monte Carlo Simulations in Statistical Physics*, 4th ed., Cambridge UP (2014).
- U. Wolff, "Collective Monte Carlo updating for spin systems", *Phys. Rev. Lett.* **62**, 361 (1989).
- R. H. Swendsen and J.-S. Wang, "Nonuniversal critical dynamics in Monte Carlo simulations", *Phys. Rev. Lett.* **58**, 86 (1987).
- K. Hukushima and K. Nemoto, "Exchange Monte Carlo method and application to spin glass simulations", *J. Phys. Soc. Jpn.* **65**, 1604 (1996).
- H. Katzgraber, S. Trebst, D. Huse, M. Troyer, "Feedback-optimized parallel tempering Monte Carlo", *J. Stat. Mech.* P03018 (2006).
- U. Drepper, "What every programmer should know about memory", `lwn.net` (2007). *Free, classic, and still essentially correct—the canonical reference for the cache-hierarchy facts above.*
- R. F. L. Evans *et al.*, "Atomistic spin model simulations of magnetic nanomaterials", *J. Phys.: Condens. Matter* **26**, 103202 (2014). *The* Vampire *paper.*